Lab 6 Report: Single-Cycle CPU

Jatin Punjabi, EC413

1 Overview

We extended the supplied single-cycle MIPS CPU (Verilog) with five instructions: SLT, ADDI, J, BNE and LUI. Each instruction was added and tested on its own, and after each change we reran the earlier testbenches to make sure nothing broke. Four provided files were modified (`ALU.v`, `ALU_control.v`, `control.v` and `cpu.v`); every other module is unchanged.

Instruction	Opcode / funct	Files changed	Datapath hardware added
SLT	`000000` / `0x2A`	`ALU.v`, `ALU_control.v`	none
ADDI	`001000`	`control.v`	none
J	`000010`	`control.v`, `cpu.v`	jump address and PC multiplexer
BNE	`000101`	`control.v`, `cpu.v`	PC-select logic
LUI	`001111`	`control.v`, `cpu.v`	write-data multiplexer

2 Baseline design and testbench structure

Starting point

The provided CPU is a single-cycle datapath made of `InstrMem`, `control`, `nbit_register_file`, `sign_extend`, `ALU_control`, `ALU`, `Memory`, `PC` and the branch hardware (`shift_left_2` and an adder), connected in `cpu.v`. Two details of the starter code matter for the changes below.

- **ALUOp.** `control.v` uses a 2-bit `ALUOp`: `00` for R-type (the ALU function comes from the funct field), `01` for branches (subtract) and `10` for lw/sw (add).
- **ALU function codes.** The ALU takes a 3-bit `func`: 0 add, 1 subtract, 2 AND, 3 OR, 4 NOR. Function 5 was unused and is now signed less-than (SLT).

How the testbenches work

All five testbenches (`tb_slt.v`, `tb_addi.v`, `tb_j.v`, `tb_bne.v` and `tb_lui.v`) share the same structure, modeled on the provided `tb_cpu.v`.

- **Loading the program.** With `initialize` high and reset asserted, a `load` task writes each instruction into instruction memory (one per 10 ns). Then `initialize` and reset are released and the CPU runs from address 0 on a 10 ns clock.
- **Initial state.** The register file starts with $R[i] = 10i$ (for example R2 = 20 and R3 = 30). Where a test needs specific operands, the testbench overwrites registers through the hierarchy (`uut.Register_File.register_file[n]`).
- **Parking the PC.** Each program ends with an instruction that jumps to itself (or a branch to itself), so the PC stays put while the results are checked.

- **Checking.** After the program has run, a `check` task compares each result register against its expected value and prints `PASS` or `FAIL`, followed by `ALL ... TESTS PASSED` if every check succeeded. Tests that depend on the PC also check `uut.PC_out`.
- **Waveforms.** Each testbench writes a `.vcd` dump, opened in GTKWave. The screenshots below show the signals named in each section, in the time range given in the caption.

All five testbenches pass on the final design.

3 Task 2: SLT

Description

SLT (`slt rd, rs, rt`) is an R-type instruction with funct `0x2A`. It sets `rd` to 1 if `rs` is less than `rt`, comparing both as signed numbers, and to 0 otherwise. Because it is R-type, the existing control path already handles everything except the comparison itself: `RegDst`=1 selects `rd`, `ALUSrc`=0 selects register `rt` as the second operand, and `RegWrite`=1. Only the ALU and the ALU control change.

- `ALU.v`. A new function 5 returns 1 if `$signed(a) < $signed(b)` and 0 otherwise.
- `ALU_control.v`. When `ALUOp` is `00`, funct `0x2A` now selects ALU function 5.

The comparison uses `$signed`; without it, Verilog treats the operands as unsigned and a negative number would look larger than a positive one. We did not compute the result from the sign bit of $a - b$, because that is wrong when the subtraction overflows (for example `7FFFFFFF` − `FFFFFFFF`). The control unit and the datapath are unchanged.

```
// ALU.v
else if (func == 3'd5)
    out = ($signed(a) < $signed(b)) ? 32'd1 : 32'd0;

// ALU_control.v (inside ALUOp == 2'b00)
else if (instruction == 6'h2A) // SLT
    func = 3'd5;
```

Control signals

Signal	Value	Reason
`RegDst`	1	Destination register is `rd` (bits 15:11).
`ALUSrc`	0	Second ALU operand is register `rt`.
`MemtoReg`	0	Write the ALU result, not memory data.
`RegWrite`	1	The result is written to `rd`.
`MemRead`	0	No data memory read.
`MemWrite`	0	No data memory write.
`Branch`	0	Not a branch.
`BranchNE`	0	Not a branch.
`Jump`	0	Not a jump.
`LUI`	0	Not LUI.
`ALUOp`	00	ALU control decodes the funct field; funct `0x2A` gives ALU function 5 (SLT).

Table 1: Control signals for SLT.

Datapath diagram

SLT adds no hardware. The only changes are inside the ALU and the ALU control, so the datapath diagram is unchanged.

Testbench

`tb_slt.v` sets registers R1 to R8 to chosen operands, loads six SLT instructions and checks R10 to R15. The cases cover equal operands, positive versus negative, negative versus positive, and two cases where the subtraction overflows, which an implementation based on the sign bit of $a - b$ would get wrong.

Instruction	Operands	Case	Expected
SLT R10, R1, R2	$5 < 5$	equal operands	0
SLT R11, R3, R4	$5 < -3$	positive vs. negative	0
SLT R12, R4, R3	$-3 < 5$	negative vs. positive	1
SLT R13, R5, R6	7FFFFFFF < -1	subtraction overflows	0
SLT R14, R7, R8	80000000 < 1	subtraction overflows	1
SLT R15, R8, R7	$1 <$ 80000000	mixed signs	0

Simulation results

All six checks passed. In the waveform, `ALU_function` is 5 while the SLT instructions execute, `write_register` steps through `0A` to `0F`, and `ALUOut` gives 0, 0, 1, 0, 1, 0, matching the table.

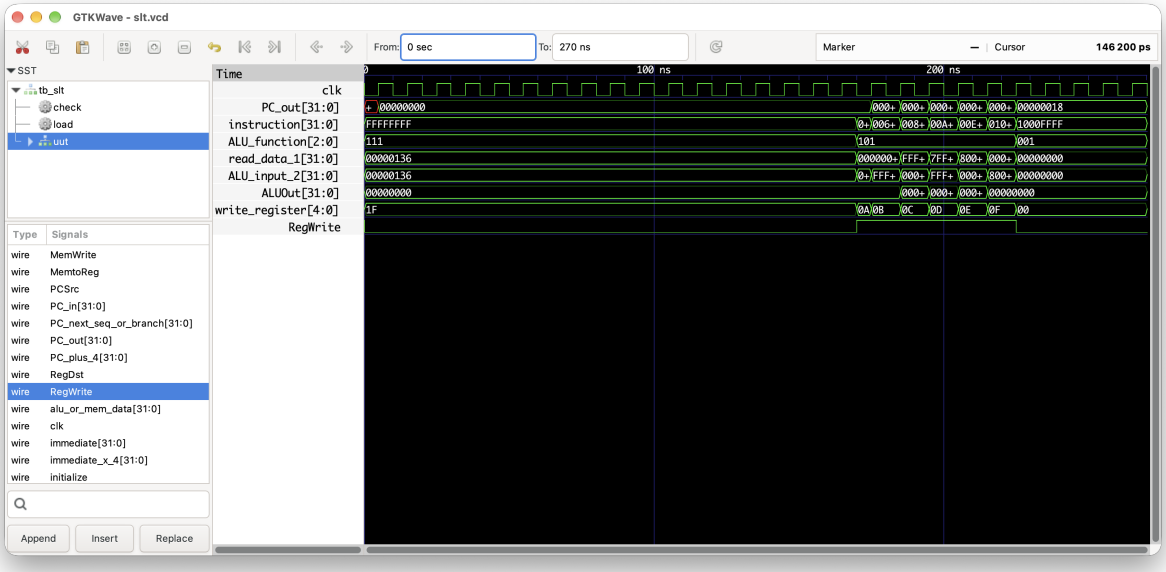


Figure 1: SLT simulation (`slt.vcd`, 170 to 270 ns).

4 Task 3: ADDI

Description

ADDI (`addi rt, rs, imm`) adds a sign-extended 16-bit immediate to `rs` and writes the result to `rt`. No new hardware is needed: the datapath already sends the sign-extended immediate to the ALU for lw and sw, so ADDI is a new decode case in `control.v` that reuses that path.

- `ALUSrc`=1 selects the immediate as the second ALU operand.
- `ALUOp`=10 makes the ALU control force an add. This matters because the low six bits of an ADDI instruction belong to the immediate, not to a funct field, and must be ignored.
- `RegDst`=0 writes `rt`, `RegWrite`=1, `MemtoReg`=0 writes the ALU result, and `MemRead`=`MemWrite`=0.

Compared with lw, the only differences are `MemRead`=0 and `MemtoReg`=0.

```
// control.v: new case, placed after lw
end else if (instruction == 6'b00_1000) begin // addi
    ALUOp = 2'b10; MemRead = 1'b0; MemtoReg = 1'b0;
    RegDst = 1'b0; Branch = 1'b0; ALUSrc = 1'b1;
    MemWrite = 1'b0; RegWrite = 1'b1;
```

Control signals

Signal	Value	Reason
`RegDst`	0	Destination register is `rt` (bits 20:16).
`ALUSrc`	1	Second ALU operand is the sign-extended immediate.
`MemtoReg`	0	Write the ALU result, not memory data.
`RegWrite`	1	The sum is written to `rt`.
`MemRead`	0	No data memory read.
`MemWrite`	0	No data memory write.
`Branch`	0	Not a branch.
`BranchNE`	0	Not a branch.
`Jump`	0	Not a jump.
`LUI`	0	Not LUI.
`ALUOp`	10	ALU control forces an add (ALU function 0) and ignores the low instruction bits.

Table 2: Control signals for ADDI.

Datapath diagram

ADDI adds no hardware; it reuses the lw immediate path, so the datapath diagram is unchanged.

Testbench

`tb_addi.v` loads five ADDI instructions and checks R1 to R5 (R0 is 0, R2 starts at 20 and R3 at 30). It covers a positive immediate, a negative immediate, adding zero, a negative result, and an instruction that uses the result of an earlier ADDI.

Instruction	Case	Expected
ADDI R1, R0, 5	positive immediate	`00000005`
ADDI R2, R2, −3	negative immediate	$20 - 3 =$ `00000011`
ADDI R3, R3, 0	adding zero	`0000001E` (unchanged)
ADDI R4, R0, −7	negative result	`FFFFFFF9`
ADDI R5, R1, 10	uses an earlier ADDI result	$5 + 10 =$ `0000000F`

Simulation results

All five checks passed. In the waveform, `ALUSrc` is high and `ALU_function` is 0 while the ADDI instructions execute, and `write_register` steps through `01` to `05` with the expected results on `ALUOut`.

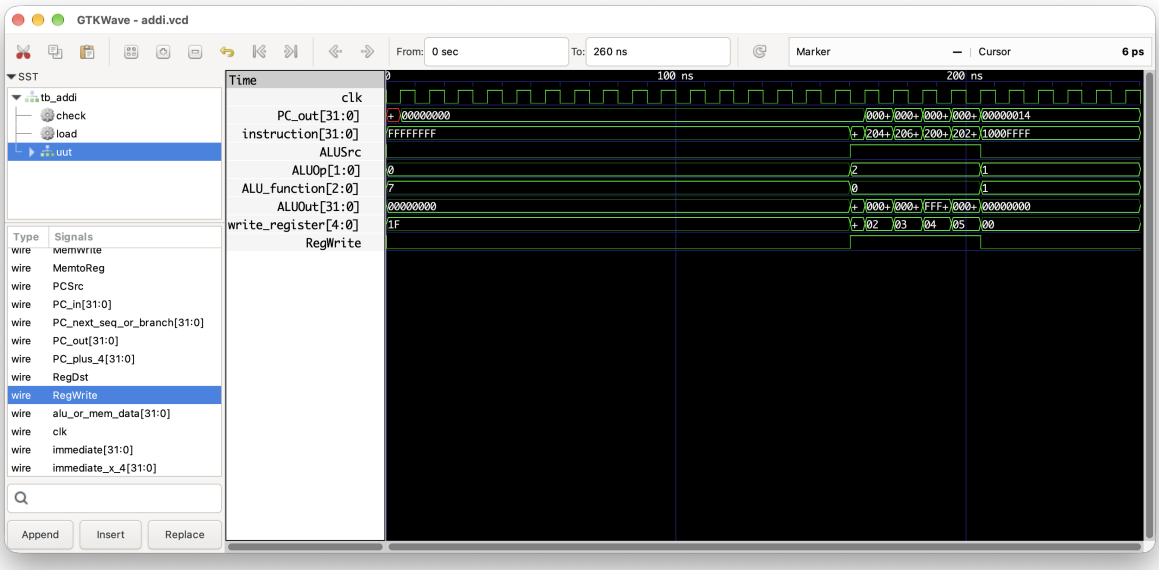


Figure 2: ADDI simulation (`addi.vcd`, 160 to 260 ns).

5 Task 4: J

Description

J (`j target`) loads the PC with a jump address built from the 26-bit target field. Following Section 4.4 of the text, the address is the upper four bits of PC+4, the 26-bit target, and two zero bits. This requires changes to both the control logic and the datapath.

- **Control (`control.v`).** A new output `Jump` is 1 only for opcode `000010` and 0 for every other instruction. For J, `RegWrite`, `MemWrite` and the branch signals are 0, so the instruction has no side effects.
- **Datapath (`cpu.v`).** The jump address is formed from `PC_plus_4[31:28]`, `instruction[25:0]` and `2'b00`. The output of the existing PC multiplexer (PC+4 or branch target, selected by `PCSrc`) now goes to a new 2-to-1 multiplexer selected by `Jump`; its other input is the jump

address, and its output drives the PC. When `Jump`=1 the jump address is selected regardless of the branch logic.

The address is built by concatenation rather than with the provided `shift_left_2`, because that module keeps its input width and a 26-bit input would lose its top two bits.

```
// cpu.v
wire [31:0] PC_next_seq_or_branch;
mux #(32) PC_Input_MUX (PCSrc, PC_plus_4, Branch_target_address, PC_next_seq_or_branch);

wire [31:0] Jump_address;
assign Jump_address = {PC_plus_4[31:28], instruction[25:0], 2'b00};
mux #(32) Jump_MUX (Jump, PC_next_seq_or_branch, Jump_address, PC_in);
```

Control signals

Signal	Value	Reason
`Jump`	1	Selects the jump address as the next PC.
`RegWrite`	0	J writes no register.
`MemWrite`	0	No data memory write.
`MemRead`	0	No data memory read.
`Branch`	0	Not a branch.
`BranchNE`	0	Not a branch.
`LUI`	0	Not LUI.
`RegDst`	0	Not used (nothing is written).
`ALUSrc`	0	Not used.
`MemtoReg`	0	Not used.
`ALUOp`	00	Not used; the ALU result is ignored.

Table 3: Control signals for J.

Testbench

`tb_j.v` runs a program that jumps over two instructions, then checks that the jump happened and the skipped instructions did not execute. R2 starts at 20 and R4 at 40, so if either skipped ADDI ran, the value would change.

Addr	Instruction	Expected
0	ADDI R1, R0, 1	R1 = 1
4	J 16 (target field 4)	PC goes to 16
8	ADDI R2, R0, 99	skipped: R2 stays 20
12	ADDI R4, R0, 77	skipped: R4 stays 40
16	ADDI R3, R0, 5	R3 = 5
20	J 20 (target field 5)	PC parks at 20

Simulation results

All five checks passed (R1, R2, R4, R3 and the final PC). In the waveform, `PC_out` goes `00`, `04`, `10`, `14`: it never takes the values `08` or `0c`, and `Jump` is high during each J instruction. `PCSrc` stays low,

because the jump is handled by the new multiplexer, and `RegWrite` is low during the jump.

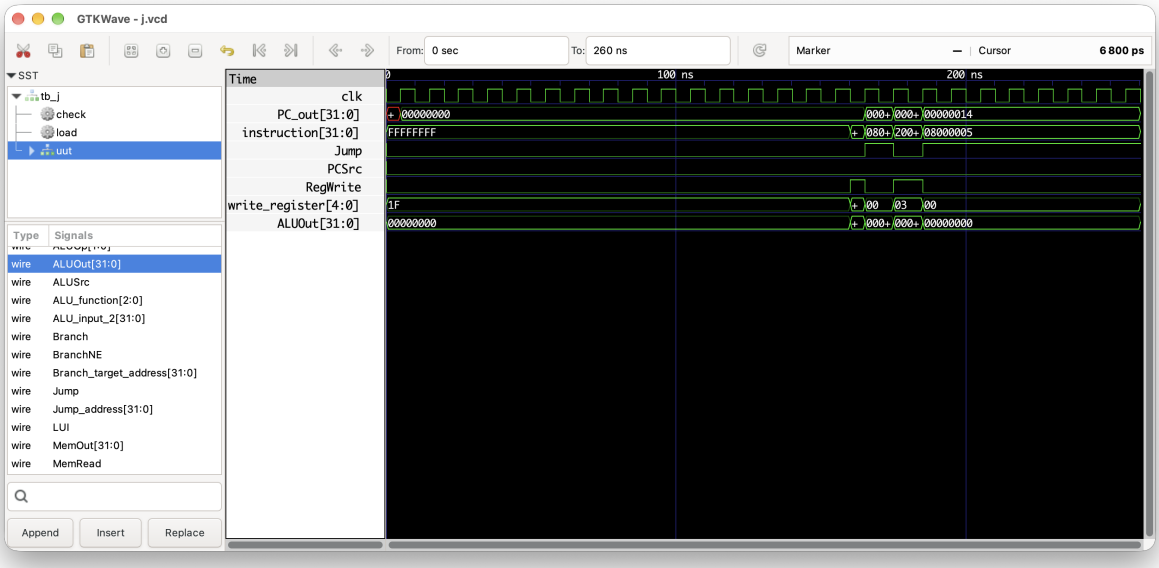


Figure 3: J simulation (`j.vcd`, 160 to 250 ns).

6 Task 5: BNE

Description

BNE (`bne rs, rt, offset`) branches when `rs` and `rt` are not equal. It uses the same subtraction and branch-target hardware as beq, so no new components are needed; only the condition that selects the branch target changes.

- **Control (`control.v`).** A new output `BranchNE` is 1 only for opcode `000101`. BNE uses `ALUOp`=01 (subtract) like beq, so `zero_flag` is 1 exactly when the operands are equal. `Branch` is 0 for BNE and `BranchNE` is 0 for beq, so they are never asserted together.
- **Datapath (`cpu.v`).** The `and` gate that produced `PCSrc` is replaced by the expression below. The branch-target adder, the shift and the PC multiplexer are unchanged.

$$\text{PCSrc} = (\text{Branch} \wedge \text{Zero}) \vee (\text{BranchNE} \wedge \neg\text{Zero})$$

```
// cpu.v
assign PCSrc = (Branch & zero_flag) | (BranchNE & ~zero_flag);
```

Control signals

Signal	Value	Reason
`BranchNE`	1	Take the branch when the ALU result is non-zero.
`Branch`	0	Only beq uses this signal.
`ALUSrc`	0	Compare two registers, not a register and an immediate.
`ALUOp`	01	ALU control selects subtract (ALU function 1), so Zero means equal.
`RegWrite`	0	A branch writes no register.
`MemWrite`	0	No data memory write.
`MemRead`	0	No data memory read.
`Jump`	0	Not a jump.
`LUI`	0	Not LUI.
`RegDst`	0	Not used.
`MemtoReg`	0	Not used.

Table 4: Control signals for BNE.

Testbench

`tb_bne.v` runs one program that covers a taken branch, a not-taken branch, a forward branch and a backward branch (a loop). R2 starts at 20, R3 at 30 and R6 at 60.

Addr	Instruction	Expected behavior
0	ADDI R1, R0, 1	R1 = 1
4	BNE R1, R0, +1	taken, forward, to 12
8	ADDI R2, R0, 99	skipped: R2 stays 20
12	BNE R0, R0, +1	not taken (operands equal), falls through
16	ADDI R3, R0, 5	executes: R3 = 5
20	ADDI R5, R0, 3	loop counter = 3
24	ADDI R5, R5, −1	loop top
28	ADDI R6, R6, 1	counts iterations: 60 to 63
32	BNE R5, R0, −3	taken, backward, to 24 (twice)
36	ADDI R7, R0, 9	after the loop: R7 = 9
40	J 40	PC parks at 40

Simulation results

All seven checks passed. In the waveform, `PC_out` runs `00`, `04`, `0c` (the instruction at `08` is skipped), `10`, `14`, then the loop `18`, `1c`, `20` three times, then `24` and finally `28`. `BranchNE` is high during each BNE. `PCSrc` goes high for the first BNE (1 and 0 differ) and for the first two loop iterations (R5 is 2, then 1), and stays low when `zero_flag` is 1: for the BNE with equal operands, and on the third iteration when R5 reaches 0. To confirm that the testbench can catch a faulty BNE, we removed the `BranchNE` term from the `PCSrc` expression in a scratch copy; the testbench then reported 3 failures.

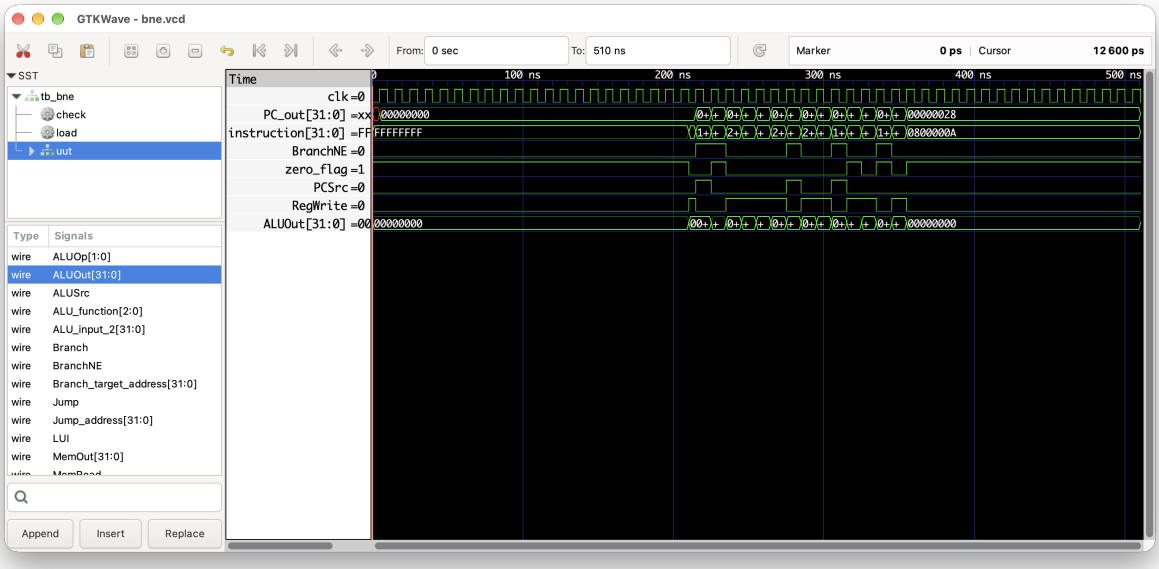


Figure 4: BNE simulation (`bne.vcd`, 210 to 380 ns).

7 Task 6: LUI

Description

LUI (`lui rt, imm`) loads the 16-bit immediate into the upper half of `rt` and clears the lower half. The shifted value is built directly from the instruction bits as {instr[15:0], 16′b0}, and a new multiplexer in front of the register file's write-data input chooses between it and the existing write-data value.

- **Control (`control.v`).** A new output `LUI` is 1 only for opcode `001111`. For LUI, `RegWrite`=1 and `RegDst`=0 (destination is `rt`); the memory and branch signals are 0, and the ALU result is not used.
- **Datapath (`cpu.v`).** The output of the existing `MemtoReg` multiplexer now feeds the new multiplexer, selected by `LUI`; its other input is the shifted immediate, and its output is the register write data.

Generating the value directly from the instruction word avoids using the ALU or the sign extender, and keeps the change to a single multiplexer.

```
// cpu.v
wire [31:0] alu_or_mem_data;
mux #(32) ALU_Mem_Select_MUX (MemtoReg, ALUOut, MemOut, alu_or_mem_data);

wire [31:0] lui_value;
assign lui_value = {instruction[15:0], 16'b0};
mux #(32) LUI_MUX (LUI, alu_or_mem_data, lui_value, write_data);
```

Control signals

Signal	Value	Reason
LUI	1	Selects the shifted immediate as the register write data.
RegWrite	1	The result is written to rt.
RegDst	0	Destination register is rt (bits 20:16).
MemtoReg	0	Passes the ALU result through the first multiplexer; it is then overridden by LUI.
MemRead	0	No data memory read.
MemWrite	0	No data memory write.
Branch	0	Not a branch.
BranchNE	0	Not a branch.
Jump	0	Not a jump.
ALUSrc	0	Not used; the ALU result is ignored.
ALUOp	00	Not used; the ALU result is ignored.

Table 5: Control signals for LUI.

Testbench

tb_lui.v loads five LUI instructions with immediates that include mixed bit patterns, all ones, all zeros and a set top bit, and checks R1 to R5. It also checks explicitly that the low 16 bits of the results are zero. R4 starts at 40, so the zero-immediate case also shows that LUI overwrites the old value.

Instruction	Case	Expected
LUI R1, 0xA5C3	mixed 0s and 1s	A5C30000
LUI R2, 0x5A5A	mixed, opposite pattern	5A5A0000
LUI R3, 0xFFFF	all ones	FFFF0000
LUI R4, 0x0000	all zeros	00000000
LUI R5, 0x8001	top bit set	80010000

Simulation results

All six checks passed (five results plus the low-16-bits check). In the waveform, LUI is high and MemtoReg is low while the LUI instructions execute, RegWrite is high, and write_data shows each shifted immediate with write_register giving its destination (01 to 05).

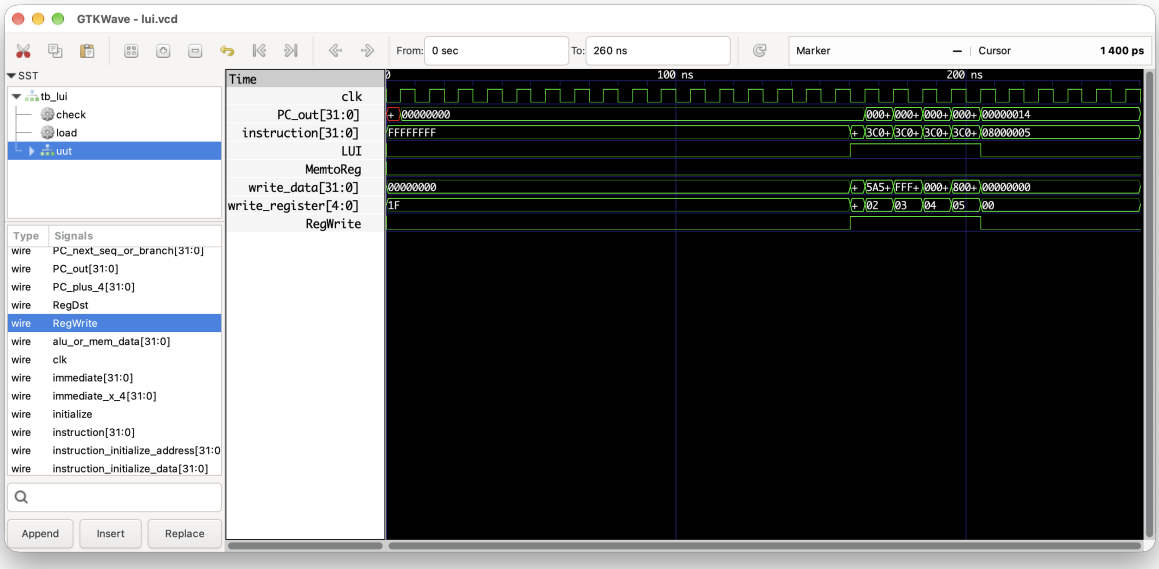
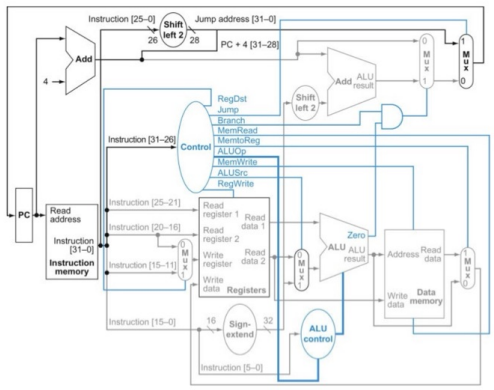


Figure 5: LUI simulation (`lui.vcd`, 160 to 250 ns).

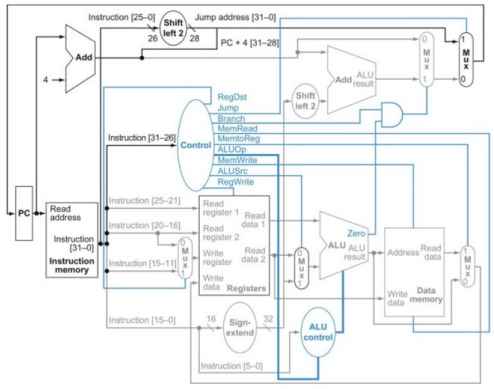
8 Appendix

Datapath Diagrams

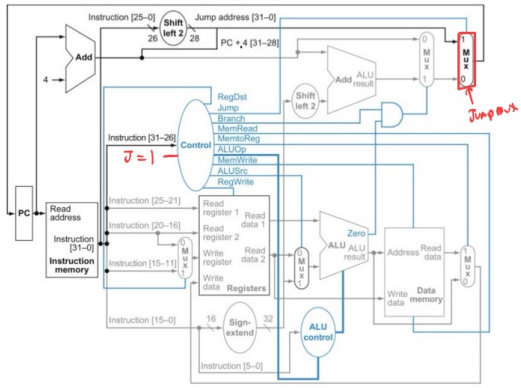
SLI - No new hardware



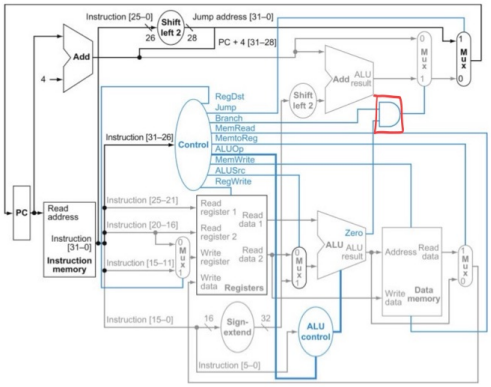
ADDI - no new hardware

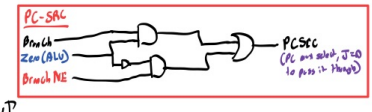


Jump



BNE





LUI

